

LAB EXPERIMENTS

Q20) Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

a. Are any blocks beyond P2 affected?

b. Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

Aim:

To write a C program to demonstrate the effect of transmission errors in **ECB (Electronic Codebook)** and **CBC (Cipher Block Chaining)** modes and analyze error propagation.

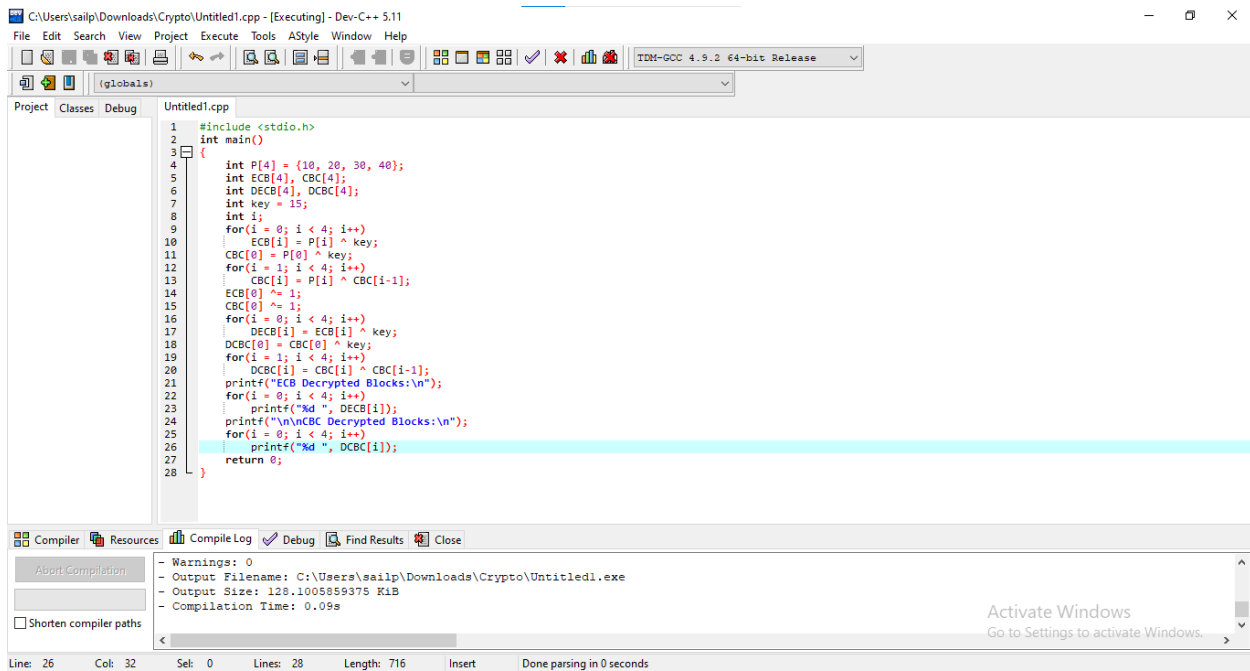
Algorithm:

1. Start the program.
2. Initialize four plaintext blocks and a secret key.
3. Encrypt each block independently for **ECB mode** using the XOR operation.
4. Encrypt each block for **CBC mode**, where each plaintext block is XORed with the previous ciphertext block before encryption.
5. Introduce a transmission error (bit flip) in the first ciphertext block.
6. Decrypt the modified ciphertext in **ECB mode** and observe the affected plaintext blocks.
7. Decrypt the modified ciphertext in **CBC mode** and observe the affected plaintext blocks.
8. Display the decrypted plaintext for both modes.
9. Compare the error propagation in ECB and CBC.
10. Stop the program.

Result:

Thus, the C program demonstrating **ECB** and **CBC** modes was successfully executed. It was observed that **ECB affects only the corresponding plaintext block**, whereas **CBC propagates the error to the next plaintext block as well**.

Code:



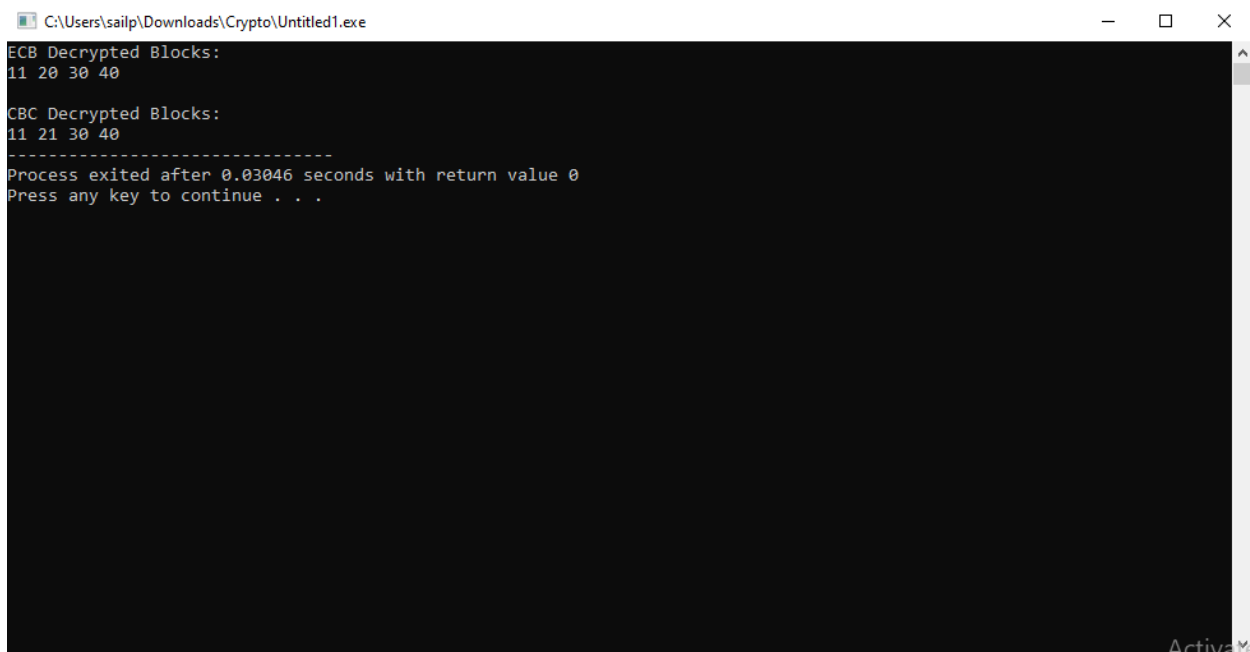
The screenshot shows the Dev-C++ IDE with the file 'Untitled1.cpp' open. The code implements a simple encryption/decryption program using XOR. It defines a plaintext array 'P' with values {10, 20, 30, 40} and a key of 15. It then calculates ciphertext blocks (ECB) and decrypts them back to the original plaintext (DCBC). The output shows the decrypted blocks as '11 20 30 40'.

```
1 #include <stdio.h>
2 int main()
3 {
4     int P[4] = {10, 20, 30, 40};
5     int ECB[4], CBC[4];
6     int DCBC[4], DCBC[4];
7     int key = 15;
8     int i;
9     for(i = 0; i < 4; i++)
10         ECB[i] = P[i] ^ key;
11     CBC[0] = P[0] ^ key;
12     for(i = 1; i < 4; i++)
13         CBC[i] = P[i] ^ CBC[i-1];
14     ECB[0] ^= 1;
15     CBC[0] ^= 1;
16     for(i = 0; i < 4; i++)
17         DCBC[i] = ECB[i] ^ key;
18     DCBC[0] = CBC[0] ^ key;
19     for(i = 1; i < 4; i++)
20         DCBC[i] = CBC[i] ^ CBC[i-1];
21     printf("ECB Decrypted Blocks:\n");
22     for(i = 0; i < 4; i++)
23         printf("%d ", DCBC[i]);
24     printf("\n\nCBC Decrypted Blocks:\n");
25     for(i = 0; i < 4; i++)
26         printf("%d ", DCBC[i]);
27     return 0;
28 }
```

Compiler Output:

```
- Warnings: 0
- Output Filename: C:\Users\sailp\Downloads\Crypto\Untitled1.exe
- Output Size: 128.1005859375 KiB
- Compilation Time: 0.09s
```

Output:



The screenshot shows the command prompt output of the program. It displays the decrypted blocks for both ECB and CBC modes, which are '11 20 30 40'. The program then exits with a return value of 0 and prompts the user to press any key to continue.

```
C:\Users\sailp\Downloads\Crypto\Untitled1.exe
ECB Decrypted Blocks:
11 20 30 40

CBC Decrypted Blocks:
11 21 30 40

-----
Process exited after 0.03046 seconds with return value 0
Press any key to continue . . .
```